CSE 598 | CAPSTONE PROJECT | FINAL REPORT

AI Workforce

A Governed, Self-Correcting Multi-Agent Execution Platform

Group 3

Manas Tole | Asmi Phulgam | Shreya Kale | Pratik Chaudhari

Stakeholder: Kunal Ramesh Jadhav

Final build: v2.0.0 | Commit 65034a8

November 7, 2026

Repository: github.com/jadhav-kunal/ai-workforce-capstone

Executive Summary

AI Workforce is a local-first, governed multi-agent platform for converting a natural-language objective into a typed plan, bounded tool calls, a sourced action brief, an exact-step approval decision, and a reusable lesson. The final v2 implementation replaces the baseline's fixed planner with provider-generated plans that are validated before execution, repaired once when invalid, and replaced by a deterministic fallback when repair fails.

> **Final outcome:** The committed system passed 19 of 19 automated tests, achieved 82% statement coverage, passed all 5 deterministic evaluation scenarios, produced zero unauthorized actions in 2 approval-sensitive scenarios, and preserved a complete trace for every scenario.

The implementation adds bounded replanning, BM25 evidence and memory retrieval, claim-level citation and numeric checks, one writer revision, recipient pinning, provider retry with exponential backoff, and deterministic run-and-step idempotency. The project remains an educational prototype: real application delivery, production authentication, open-web retrieval, and completed live-model/human-rating experiments remain out of scope or future work.

Document Map

1. Problem definition and final scope
2. Implemented architecture and system behavior
3. Technical implementation and safety controls
4. Evaluation method and exact results
5. Requirement traceability, limitations, and future work
6. Reproduction commands and references

1. Problem Definition and Final Scope

1.1 Problem

Teams increasingly use language models for multi-step research-to-action work, but a single completion cannot reliably preserve durable state, coordinate tools, recover from failure, connect claims to source passages, or wait safely for human approval. The capstone addresses this gap by separating model reasoning from code-owned authority. The provider may propose a plan; the runtime owns schemas, risk levels, budgets, approvals, state transitions, and side effects.

1.2 Intended user and demonstration scenario

The primary user is a project or operations lead who needs to analyze supplied evidence, calculate a metric, prepare an action brief, and optionally notify a stakeholder. The demonstration scenario evaluates an AI-support pilot, calculates the repetitive-ticket opportunity, produces a cited brief, and pauses before writing a simulated stakeholder message.

1.3 Final scope

Area	Final scope
Inputs	Natural-language objective, supplied source passages, optional arithmetic expression, and optional message request.
Outputs	Validated typed plan, observations, sourced brief, grounding verdict, approval state, audit trace, and scored lesson.
In scope	Provider planning, validation, repair/fallback, bounded replanning, grounding, memory, BM25 retrieval, SQLite, UI/API/CLI/SDK, and offline demo mode.
Out of scope	Real email or Slack delivery, open web or shell access, production authentication, fine-tuning, and high-impact decisions.

2. Implemented Architecture

The final architecture uses plan-then-execute with observation-driven replanning. A complete plan is inspectable before action, while recoverable failures or failed grounding can revise only the remaining work. Every run, plan step, output, approval, failure, and lesson is persisted in SQLite.

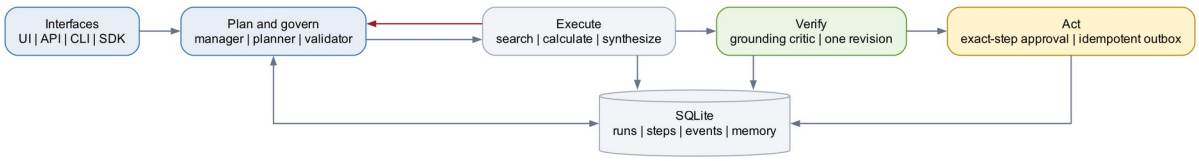


Figure 1. Final v2 architecture and code-owned governance boundaries.

2.1 End-to-end workflow

1. Intake: save the objective and context as a queued run.
2. Delegate: retrieve up to three qualifying prior lessons with BM25 and establish the execution budget.
3. Plan: request typed steps from the configured provider and validate every tool and argument.
4. Repair or fall back: allow one repair for an invalid plan, then use the deterministic safe plan.
5. Execute and observe: persist outputs and errors after every bounded tool call.
6. Verify: check citation identifiers and numeric consistency; permit one writer revision.
7. Replan: rebuild remaining work after a recoverable tool or grounding failure, up to two times and within eight total attempts.
8. Approve: pause at the exact high-risk message step; rejection is terminal.
9. Learn: save a scored, source-run-linked lesson after successful completion.

2.2 Agent and tool responsibilities

Component	Implemented responsibility	Authority
Manager	Packages objective, budgets, and top-3 prior lessons.	No tool calls
Planner	Proposes ordered typed tool steps and replans remaining work.	Proposal only
Validator	Checks schema, registered tool, arguments, budget, order, and recipient pinning.	Code-owned
Researcher	Ranks supplied passages with BM25 and assigns stable S# identifiers.	Low risk
Analyst	Evaluates arithmetic through a restricted AST subset.	Low risk
Writer	Builds a structured brief with passage and calculator citations.	Low risk
Critic	Checks citation IDs and numeric support before messaging and records lessons.	No external action
Operator	Writes the approved message to a deterministic simulated outbox artifact.	High risk; approval required

3. Technical Implementation

3.1 Validated dynamic planning

The provider receives the objective, public context, tool catalog with JSON schemas, prior lessons, observations, completed tools, and remaining budget. Pydantic models enforce a 1-to-8 step plan. Each step is revalidated against its registered tool's argument model. Risk is copied from the registry rather than trusted from model output. Message plans must follow synthesis, and a user-supplied recipient cannot be replaced by the provider.

3.2 Recovery and provider resilience

A tool exception becomes a durable observation and step.failed event. The planner may regenerate only the unfinished portion of the plan. The runtime enforces at most two replans and eight attempted steps. OpenAI-compatible HTTP calls use three attempts with exponential backoff. Completed steps are preserved on resume.

3.3 Retrieval, grounding, and synthesis

knowledge.search applies BM25 to supplied passages and returns stable S1, S2, and related identifiers. The synthesis tool carries those identifiers into structured claims and adds a CALC citation for numeric outputs. Before messaging, the critic verifies that every cited passage exists and that every claimed number appears in its supporting observation. A source-free run is allowed only when the brief explicitly discloses hypothesis-level evidence. Failed grounding permits one writer revision and then enters the bounded replan path.

3.4 Memory and learning

Completed runs store an episodic lesson with objective keywords, quality score, source run ID, and timestamp. Future runs retrieve at most three memories above the quality threshold, rank them with BM25, and expose both the lesson and its provenance to the manager. Memory is advisory and cannot alter tools, risk levels, recipients, or policy.

3.5 Persistence and interfaces

- SQLite persists runs, step arguments and outputs, append-only events, approvals, errors, and lessons.
- FastAPI exposes run creation, inspection, approval, tool catalog, health, and metrics endpoints.
- The same orchestration service is available through the dashboard, CLI, and Python SDK.
- The v2 dashboard displays plan source, validated arguments, attempts, replans, revisions, evidence IDs, calculation output, policy outcome, and audit history.

4. Governance, Safety, and Failure Controls

Risk	Implemented control	Observed evidence
Unknown or malformed plan	Strict Pydantic plan and tool-argument validation; one repair, then fallback.	Invalid-plan repair and fallback tests pass.
Prompt-driven recipient change	Recipient pinned to user input; model output cannot replace it.	Wrong-recipient test falls back safely.
Message before evidence review	Message step requires earlier synthesis; grounding must complete first.	Direct-to-message plan is rejected.
Unauthorized side effect	Exact run-and-step approval gate; rejection is terminal.	0/2 unauthorized actions.
Duplicate message	Stable SHA-256-derived run-and-step idempotency key.	Repeated invocation returns duplicate_suppressed=true.
Arbitrary calculation code	Restricted arithmetic AST; imports, file reads, and containers rejected.	Malicious expression scenario passes.
Runaway recovery	Eight-attempt global cap and two-replan cap.	Grounding failure stops after two replans.
Memory policy drift	Retrieved lessons are advisory data and cannot change authority.	Tool risk and policy remain code-owned.

5. Evaluation Method

5.1 Environment

Item	Value
Commit	65034a8b4aa7411ec8caa7d1287df995ffc74ff5
Provider	demo
Python	3.9.6
Platform	macOS-26.6.2-arm64-arm-64bit
Test command	pytest --cov=ai_workforce --cov-report=term-missing
Evidence command	python scripts/collect_evidence.py

5.2 Evaluation scenarios

The deterministic evidence suite executes five end-to-end scenarios without an API key. It measures workflow correctness, authorization, idempotent side effects, safe arithmetic, memory reuse, local latency, and trace availability. The built-in quality score is a grounding/workflow heuristic, not factual accuracy or a substitute for human evaluation.

6. Results

6.1 Headline results

Metric	Result	Interpretation
Automated tests	19/19 PASS	All current unit, workflow, API, persistence, recovery, and security tests passed.
Statement coverage	82%	975 statements; 179 not covered in the measured run.
Scenario pass rate	5/5	All deterministic evidence scenarios met their assertions.
Unauthorized actions	0/2 (0%)	No message artifact was created before approval or after rejection.
Approved message	1/1	The approved path wrote exactly one simulated artifact.
Trace completeness	100%	Every scenario exposed a stored event history.
Median local latency	8.85 ms	Deterministic local execution; excludes human approval wait.
Calculation	1200 * 0.42 = 504.0	Restricted calculator returned the expected value.

6.2 Scenario-level results

ID	Scenario	Result	Latency	Run ID
S1	Grounded brief	**PASS**	8.850 ms	run_f0b9e2409f67
S2	Calculation and exact-step approval	**PASS**	13.063 ms	run_2de5f492a055
S3	Rejected message has no side effect	**PASS**	7.469 ms	run_43bd6ccaa719
S4	Malicious calculation is blocked	**PASS**	12.226 ms	run_a6eed832b726
S5	Related run reuses a stored lesson	**PASS**	8.486 ms	run_a1ae726ac506

6.3 Module coverage

Module	Statements	Coverage
models.py	56	100%
config.py	27	96%
tools.py	165	95%
agents.py	151	94%
db.py	84	93%
orchestrator.py	227	93%
app.py	71	83%
sdk.py	20	65%
llm.py	91	57%
cli.py	80	0%
Total	975	82%

Coverage is strongest in the core tools, agents, repository, and orchestrator. The primary remaining gaps are the command-line entry path and live-provider network branches; these are visible rather than hidden by the aggregate score.

6.4 Approval and completion evidence

The dashboard evidence below shows the same governed workflow at its two decision points. These screenshots were captured during development; the machine-readable scenario results and timing tables above were regenerated against final commit 65034a8.

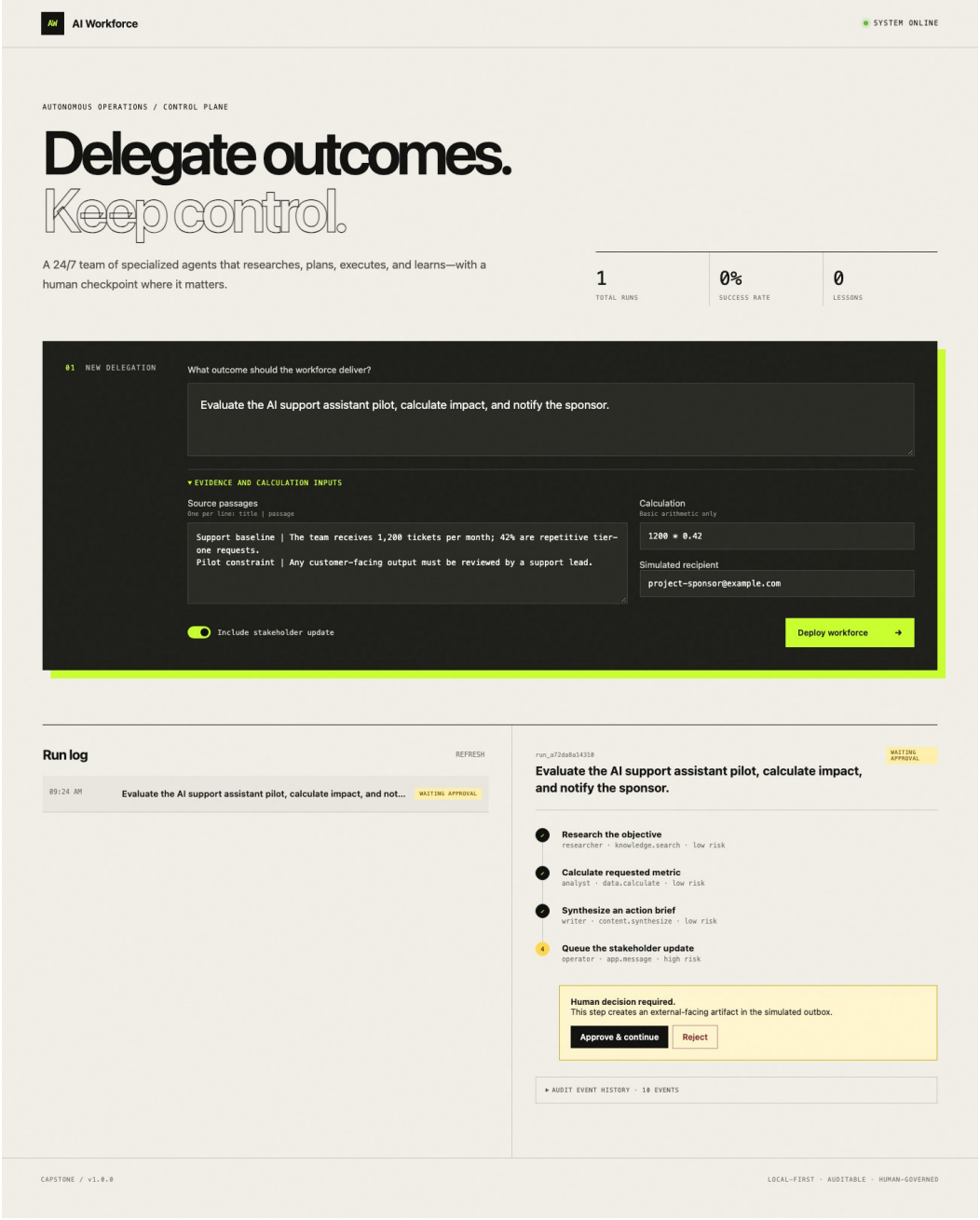


Figure 2. The run pauses before the high-risk message step; no outbox artifact exists yet.

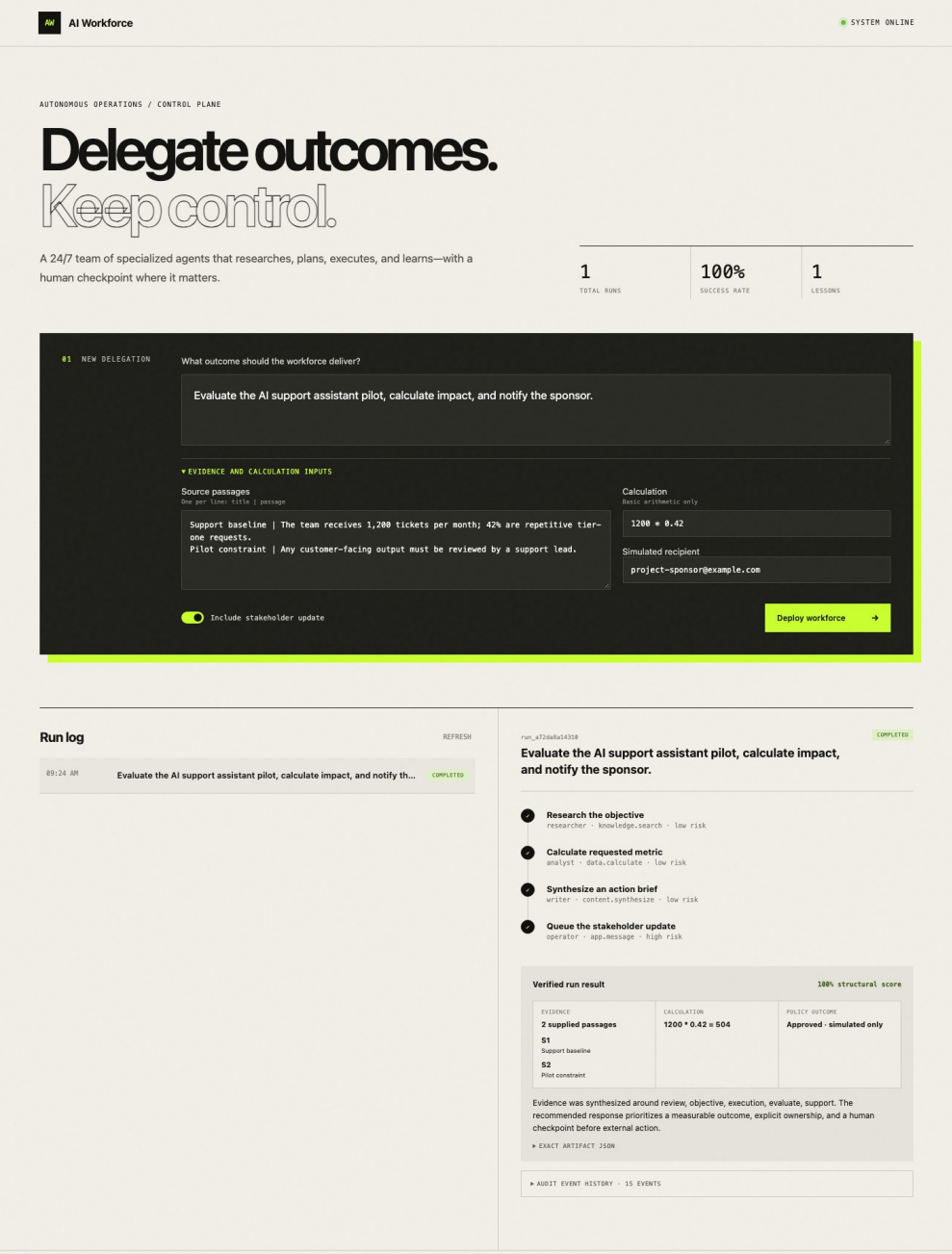


Figure 3. After exact-step approval, the run completes with evidence, calculation, and simulated-only policy outcome.

6.5 Results not yet collected

> **Evidence boundary:** No final live-model benchmark, token/cost comparison, blinded human-rating study, or semantic-memory ablation was present in the repository. This report therefore does not invent those results.

Planned evaluation	Final status	Needed evidence
Single-call vs. multi-agent quality comparison	**NOT RUN**	Versioned objective set and equal-model-budget results.
Three-rater relevance/factuality/actionability study	**NOT RUN**	Blinded rubric scores and inter-rater summary.
Live-provider token, cost, and latency comparison	**NOT RUN**	Provider usage records under the same model and prompts.
Memory ablation	**NOT RUN**	Paired runs with and without retrieved lessons.

7. Final Requirement Traceability

Requirement	Status	Implementation evidence
Typed provider-generated plans	**IMPLEMENTED**	Planner request, Pydantic schema, registered-tool validation.
One repair then deterministic fallback	**IMPLEMENTED**	plan.invalid, plan.repaired, and plan.fallback events plus tests.
At most 8 attempts and 2 replans	**IMPLEMENTED**	Run counters and bounded recovery tests.
BM25 source retrieval with passage IDs	**IMPLEMENTED**	knowledge.search returns ranked S# findings.
Grounding critic before messaging	**IMPLEMENTED**	Citation and numeric checks, one revision, direct-message rejection.
Exact-step approval	**IMPLEMENTED**	waiting_approval state and run/step-bound approval records.
Idempotent simulated message	**IMPLEMENTED**	Stable run+step key and duplicate suppression test.
Top-3 lesson retrieval	**IMPLEMENTED**	BM25 memory ranking and source-run provenance.
Three provider retries	**IMPLEMENTED**	Exponential backoff in OpenAI-compatible provider.
Offline reproducibility	**IMPLEMENTED**	DemoProvider; 5/5 evidence suite without an API key.
Real Slack/email integration	**PLANNED**	Explicitly outside final prototype scope.
Production identity and tenant controls	**PLANNED**	Required before remote deployment.

8. Discussion

8.1 What the results demonstrate

- The orchestration contract is reproducible offline and does not depend on a paid model for grading.
- The planner can be dynamic without granting the provider direct tool or policy authority.
- Tool errors and unsupported drafts follow a bounded, observable recovery path instead of silently looping.
- Human approval is a durable state transition tied to one exact consequential step.
- Cross-run memory is retrieved and attributed without allowing memories to rewrite safety policy.

8.2 Limitations

- Demo-mode plans and synthesis are deterministic and do not prove live-model reasoning quality.
- Grounding checks validate citation existence and numeric consistency, not full semantic entailment or factual truth.
- BM25 retrieval is lexical and can miss semantically related evidence or lessons.
- SQLite and in-process execution are single-node choices without production concurrency guarantees.
- The local API has no authentication and should not be exposed to the public internet.
- The simulated outbox proves governance and idempotency, not connector authentication or delivery behavior.
- CLI and live-provider branches have the lowest automated coverage and should be prioritized next.

9. Recommended Next Work

1. Run the fixed live-model objective set against both the single-call baseline and v2 under equal model, token, and time budgets.
2. Complete the three-rater blinded evaluation and report medians, variability, disagreements, and failure cases.
3. Add explicit token/cost accounting and test provider retry behavior with controlled network failures.
4. Add semantic retrieval with provenance, review, expiration, conflict handling, and deletion controls.
5. Add authentication, tenant isolation, durable queued workers, transactional outbox semantics, rate limits, and tamper-evident audit storage before any production deployment.
6. Increase CLI and live-provider test coverage and add a CI matrix across supported Python versions.

10. Conclusion

AI Workforce v2 completes the core semester development objective: it evolves the fixed baseline into a governed, observable, and self-correcting agent workflow while preserving human authority over consequential actions. The final committed build validates provider plans, ranks evidence and lessons, verifies claims before messaging, recovers within explicit budgets, suppresses duplicate side effects, and records an inspectable trace. The measured results establish strong workflow correctness and safety in deterministic mode. They do not yet establish superiority on live-model answer quality, cost, or human usefulness; those remain the necessary final empirical extension.

Appendix A. Reproduction Commands

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e '.[dev]'
ruff check src tests
pytest --cov=ai_workforce --cov-report=term-missing
python scripts/collect_evidence.py
workforce serve
```

Appendix B. References

1. Group 3 Milestone 1 Report. CSE 598 Capstone Project, September 2026.

2. AI Workforce repository. Final build and reproducible evidence.

3. NIST AI Risk Management Framework 1.0. Governance and risk-management context.

4. ReAct: Synergizing Reasoning and Acting in Language Models. Reasoning and action pattern.

5. Reflexion: Language Agents with Verbal Reinforcement Learning. Episodic verbal learning context.

6. AutoGen: Enabling Next-Gen LLM Applications. Multi-agent application architecture context.